

TEMA 9

ADMINISTRACIÓN EN LINUX II

1	Comandos para la gestión de ficheros, directorios y discos.	2
1.1	Propietarios y permisos de ficheros y directorios.	2
1.1.1	Comando CHOWN.....	3
1.1.2	Comando CHGRP	4
1.1.3	Comando CHMOD.....	4
1.1.4	Comando UMASK	5
1.2	Comandos para empaquetar, comprimir y descomprimir archivos	6
1.2.1	Comando GZIP	6
1.2.2	Comando BZIP2	6
1.2.3	Comando TAR.....	7
1.3	Comandos para el manejo de discos y particiones.....	8
1.3.1	Comando FDISK.	9
1.3.2	Comando MKFS.....	10
1.3.3	Comando MOUNT.	10
1.3.4	Comando FSCK.....	12
2	Comandos para la gestión de procesos	13
2.1	Procesos en Linux	13
2.2	Redireccionamiento.	15
2.3	Comandos sobre procesos.....	16
2.3.1	Comando PS	16
2.3.2	Comando PSTREE	17
2.3.3	Comando NICE	17
2.3.4	Comando RENICE	17
2.3.5	Comando JOBS	18
2.3.6	Comando FG.....	18
2.3.7	Comando BG	18
2.3.8	Comando KILL.....	18
3	Bibliografía.....	19

1 Comandos para la gestión de ficheros, directorios y discos.

1.1 Propietarios y permisos de ficheros y directorios.

En Linux todo es un fichero por lo que controlando quién puede acceder a ellos, podemos controlar, no sólo quién puede ejecutar cada comando o aplicación, sino también quién puede o no acceder a otros recursos como impresoras, disqueteras, unidades de CD, etc. Por tanto, la piedra angular de la seguridad en Linux, junto a la definición de usuario y grupo, van a ser los ficheros.

Cada fichero tiene asociado:

- **Un usuario propietario.** En un principio es el usuario que crea el fichero, aunque posteriormente se puede cambiar el propietario del fichero.
- **Un grupo propietario.** En un principio es el grupo activo del usuario que crea el fichero (el cual suele ser el grupo primario si no se ha cambiado), aunque posteriormente se puede cambiar el grupo propietario del fichero.
- Definición de **permisos para el usuario propietario.** Serán los permisos de que disfrutará sobre el fichero el propietario del fichero.
- Definición de **permisos para el grupo propietario.** Serán los permisos de que disfrutarán sobre el fichero los usuarios que pertenezcan al grupo propietario del fichero.
- Definición de **permisos para el resto de usuarios.** Serán los permisos de que disfrutarán sobre el fichero el resto de usuarios del sistema.

Cuando se hace un listado en formato largo (`ls -l`), aparece información relativa al usuario propietario y al grupo propietario de cada fichero.

Existen tres tipos de permisos: *permiso de lectura* (r), *permiso de escritura* (w) y *permiso de ejecución* (x), los cuales definirán qué puede hacer cada usuario con un fichero. El significado de estos permisos aplicados a ficheros o directorios se traduce en lo siguiente:

Permiso	Fichero	Directorio
r	Leerlo	Buscar en él, ver su contenido. Es decir, poder hacer un <code>ls</code> .
w	Modificarlo	Alterar su contenido. Es decir, borrar ficheros, crear directorios o ficheros, etc.
x	Ejecutarlo	Entrar en el directorio. Es decir, poder hacer un <code>cd</code> o poder ponerlo en una ruta.

Los permisos sobre un fichero y lo que cada uno permite hacer sobre él son evidentes: si se tiene permiso de lectura se puede ver lo que contiene, si se tiene permiso de escritura se puede modificar lo que contiene y si se tiene permiso de ejecución y se trata de un fichero ejecutable, se puede ejecutar. Tan sólo debemos reseñar que para poder ejecutar un proceso por lotes o *shell script*, éste tiene que tener tanto el permiso de ejecución como el permiso de lectura. Esto es debido a que un fichero de este tipo contiene comandos que deben ser leídos para poder ejecutarlos. Para ejecutar un programa ya compilado tan sólo hay que cargarlo en memoria y empezar su ejecución, para lo que basta con el permiso de ejecución.

Los permisos sobre un directorio pueden parecer un poco raros en un principio pero, mirados en profundidad, tienen sentido. Veámoslo una tabla resumen con lo más significativo:

Permiso sobre directorio	Descripción
--- (ninguno)	No se permite ninguna actividad de ningún tipo sobre el directorio, ni sobre su contenido o cualquiera de sus subdirectorios.
r-- (lectura)	Permite que se listen los nombres de los ficheros contenidos en el directorio, pero no permite mostrar información alguna sobre los atributos de dichos ficheros (ni tamaño, ni propietario, ni fecha, etc).
--x (ejecución)	Permite ejecutar programas (siempre que tenga permiso para ello en dichos programas) que el usuario previamente conoce (conoce que existen y su nombre). Esto sucede porque no se tiene permiso de lectura sobre el directorio, por lo que no podemos conocer su contenido.
r-x (lectura y ejecución)	Permite listar los nombres de los ficheros contenidos en el directorio con toda su información asociada y permite ejecutar los programas para los que se tenga permiso.
-wx (escritura)	Permite modificar el contenido del directorio; es decir, crear ficheros, borrarlos, cambiarles el nombre, etc. ¡Ojo: se pueden borrar ficheros aunque no tengamos permiso de escritura en el propio fichero! Reflexión: ¿Tiene sentido el permiso w solo; es decir -w- ?
rw x (todos)	Se tiene acceso total al directorio.

Podemos ver los permisos que tiene un fichero al realizar un listado en formato largo (comando `ls -l`). La información sobre los permisos de un fichero viene dada con el siguiente formato:

Permisos para el Usuario propietario	Permisos para el Grupo propietario	Permisos para Otros usuarios									
<table border="1"><tr><td>r</td><td>w</td><td>x</td></tr></table>	r	w	x	<table border="1"><tr><td>r</td><td>w</td><td>x</td></tr></table>	r	w	x	<table border="1"><tr><td>r</td><td>w</td><td>x</td></tr></table>	r	w	x
r	w	x									
r	w	x									
r	w	x									

NOTA: Hay otros permisos de más alto nivel (s, t, l) que no veremos.

Si un permiso está dado aparecerá la letra correspondiente en su lugar correspondiente. Por el contrario, si un permiso no está dado, aparecerá un guión (-) en el lugar correspondiente. Así, por ejemplo, la combinación `rw- r--` está reflejando los siguientes permisos:

- Permisos de lectura, escritura y ejecución para el propietario del fichero.
- Permiso de lectura y escritura para los usuarios que pertenezcan al grupo propietario del fichero.
- Permiso de lectura para el resto de usuarios.

Inmediatamente delante de los permisos va otro carácter que indica el tipo del fichero: fichero ordinario (-), directorio (d), driver de bloques (b), driver de caracteres (c).

1.1.1 Comando CHOWN

Cambia el usuario propietario de uno o varios ficheros. También puede usarse para cambiar el grupo propietario de un fichero.

El único usuario que puede cambiar el usuario propietario de un fichero es el superusuario. El superusuario tiene libertad total para establecer el usuario y el grupo propietario de cualquier fichero del sistema.

Un usuario sólo puede cambiar el grupo propietario de un fichero que le pertenezca; es decir, de aquél del que sea el usuario propietario. Además, sólo puede poner como grupo propietario de un fichero un grupo al que él mismo pertenezca.

Sintaxis:

```
chown [parametros] nuevoPropietario:nuevoGrupoPropietario fichero1
fichero2 ...
```

Parámetros

- R Cambia de forma recursiva la propiedad de los directorios y sus contenidos.
- v Describe en detalle los cambios de propiedad.
- c Describe en detalle sólo los archivos cuya propiedad cambia.
- f No imprime mensajes de error de aquellos ficheros a los que no se les pudo cambiar la propiedad.

Si sólo se desea cambiar el usuario no hace falta poner ni el punto ni el nuevo grupo. Si sólo se desea cambiar el grupo propietario hay que poner un punto seguido del nombre del grupo.

1.1.2 Comando CHGRP

Cambia el grupo propietario de los ficheros especificados. Es equivalente al anterior, pero sólo da la posibilidad de cambiar el grupo propietario. Todo lo explicado en el comando `chown` es aplicable aquí.

Sintaxis:

```
chgrp [param] nuevoGrupoPropietario fichero1 fichero2 ...
```

Parámetros

- R Cambia de forma recursiva el grupo propietario de los directorios y sus contenidos.
- v Describe en detalle los cambios de propiedad de grupo.
- c Describe en detalle sólo los archivos cuya propiedad de grupo cambia.
- f No imprime mensajes de error de aquellos ficheros a los que no se les pudo cambiar la propiedad.

1.1.3 Comando CHMOD

Cambia los permisos de acceso a uno o varios ficheros o directorios.

Sintaxis: `chmod [parametros] permisos fichero1 fichero2 ...`

Parámetros

- R Cambia de forma recursiva los permisos de los directorios y sus contenidos.
- v Describe en detalle los cambios de permisos.
- c Describe en detalle sólo los archivos cuyos permisos han cambiado.
- f No imprime mensajes de error de aquellos ficheros a los que no se les pudo cambiar los permisos.

Existen dos formas de **cambiar los permisos** de un archivo :

- **Modo simbólico.**

Existen tres niveles en el cambio de permisos, representados por: u, g, o, a. Donde:

- ‘u’ representa al propietario,
- ‘g’ representa al grupo,
- ‘o’ representa a los otros usuarios del sistema,
- ‘a’ los representa a todos

Para agregar un permiso se utiliza el signo +, para eliminar un permiso se usa el signo - y para establecer exactamente un permiso a algo se usa el signo =.

Por ejemplo, si queremos dar permiso de todo a un fichero, la forma sería alguna de estas tres:

```
chmod u+rw,g+rw,o+rw fichero
chmod a+rw fichero
chmod +rw fichero
```

Entre el último permiso de la opción anterior y la letra del siguiente grupo de permiso debe haber únicamente una coma, no puede haber ningún espacio en blanco.

También se puede dar permisos o quitar de forma selectiva, por ejemplo :

```
chmod ug+w-r,o=r fichero
```

De esta forma damos permiso de lectura y le quitamos el permiso de escritura tanto al propietario como al grupo propietario (el permiso de ejecución para ambos lo dejamos a lo que estuviera).

Además, le damos al resto sólo los permisos de lectura (retirándoles el resto de permisos).

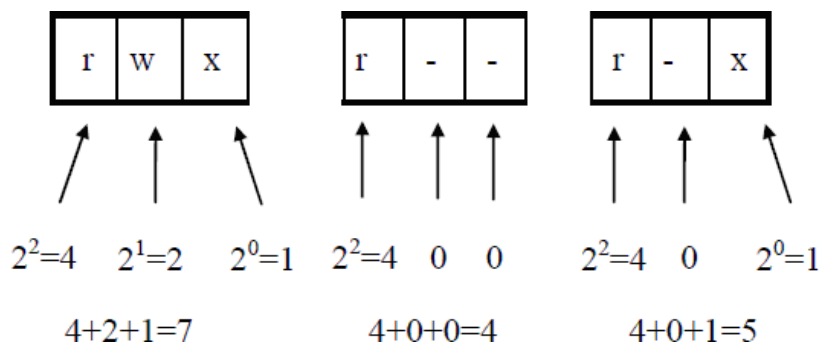
- **Modo octal o numérico.**

Consiste en dar 3 números en base octal. Cada número octal establece un byte en el campo de modalidad almacenado en la tabla de nodo-i del sistema de archivos.

Por ejemplo, supongamos que queremos darle los siguientes permisos a un fichero:

`rwxr--r-x`

Para representar estos permisos en octal tenemos que realizar los siguientes cálculos:



y habría que usar la siguiente orden: `chmod 745 fichero`

1.1.4 Comando UMASK

Este comando sirve para especificar los permisos por defecto con los que tienen que crearse los ficheros y directorios. En principio, los permisos de creación para los ficheros es de 666 (rw- rw- rw-) y para los directorios es de 777 (rwx rwx rwx). Mediante el comando `umask` podemos variar esto.

NOTA: Realmente, cuando nos conectamos al sistema ya hay una máscara establecida que hace que los permisos por defecto no sean 666 y 777 respectivamente, pero esta máscara se puede cambiar. Para que los permisos por defecto sean 666 y 777 habría que establecer una máscara de 000.

Sintaxis: `umask [d1d2d3]`

Parámetros

-S Muestra la información de forma simbólica, en vez de octal.

Si el comando es utilizado sin parámetros, lo que hace es mostrar la máscara actual. La máscara queda activa hasta que finaliza la sesión de trabajo. Como parámetro se le puede pasar la nueva máscara (tres números octales) que se quiere establecer. El cálculo de estos tres dígitos se realiza de idéntica forma que en el comando `chmod` anterior, con la única diferencia que en esta orden hay que dar el número octal de los permisos que queremos **quitar** con respecto a los permisos por defecto 666 y 777.

Por lo tanto, si establecemos como máscara 023, esto hace que los ficheros y directorios se creen con los siguientes permisos:

	Fichero	Directorio
Permisos Base (666 y 777)	r w - r w - r w -	r w x r w x r w x
Máscara 023 = 000 010 011	0 0 0 0 1 0 0 1 1	0 0 0 0 1 0 0 1 1
Permisos de creación resultantes	r w - r - - r - -	r w x r - x r - -

Debemos recordar que el permiso de ejecución sólo tiene sentido en ficheros que contienen código binario ejecutable o en shell scripts, por eso no puede ponerse bajo ningún caso como permiso por defecto para un fichero.

Algunos editores de ficheros, como el `joe` o el `vi`, tienen su propia máscara de permisos para los ficheros que crean. Si dicha máscara es más restrictiva que la del usuario que usa el editor, se ponen los permisos definidos por la máscara del editor. Por el contrario, si la máscara del usuario es más restrictiva que la del editor, se usarán los permisos del usuario.

El administrador del sistema puede establecer cuáles es la máscara por defecto de los usuarios en el sistema. Para ello tendrá que establecerlo en el fichero de configuración que corresponda.

Posteriormente, un usuario puede establecer cuál quiere que sea su máscara por defecto, cambiando su fichero de configuración correspondiente.

1.2 Comandos para empaquetar, comprimir y descomprimir archivos.

Ubuntu incluye soporte por defecto para muchos **formatos de compresión** libres (como ZIP, gzip y bzip2), sin embargo es bastante habitual encontrar archivos ACE o RAR por ejemplo, ambos formatos propietarios pero que en ocasiones nos resultan imprescindibles.

Los métodos de compresión más usados en sistemas basados en Unix son gz y bz. Estos métodos solo comprimen archivos, no directorios, es por ello que antes hay que empaquetar todo los archivos utilizando el comando tar.

Es importante aclarar que la orden del tar no comprime, solo almacena archivos y directorios en un solo fichero, por lo que no reduce el tamaño de los archivos. Sin embargo se puede combinar la funcionalidad de los archivos **.tar** con una compresión de datos que disminuya su tamaño final.

Para comprimir y descomprimir archivos están los comandos gzip y bzip2 que veremos a continuación. Otros comandos de compresión son zip, rar, lha, zoo y arj.

1.2.1 Comando GZIP

Gzip se complementa con el comando gunzip que lo que hace es descomprimir archivos comprimidos con gzip.

Sintaxis: gzip [-parametros] fichero

Parámetros

- d Descomprime un archivo comprimido con gzip.
- l Muestra información sobre el archivo comprimido que le pasemos como parametro.
- r A esta opción se le pasa como parametro un directorio. Gzip navegara por ese directorio de forma recursiva y comprimirá todos los archivos que encuentre dentro de la estructura de directorios.
- t Comprueba que un archivo comprimido con gzip este correcto. Si se encuentra bien no muestra ningún error y si lo encuentra nos informa de cual es el problema.
- v Muestra el nombre y el porcentaje de reducción por cada fichero comprimido o descomprimido.

Si queremos comprimir un archivo con gzip:

```
gzip documento
Esto crearía el archivo documento.gz. El archivo documento desaparece y se crea documento.gz que es el archivo comprimido.
```

Ahora para descomprimirlo habría que hacer lo siguiente:

```
gzip -d documento.gz
gunzip documento.gz
Esto descomprimiría el archivo documento.gz y crearía el archivo documento.
```

1.2.2 Comando BZIP2

Bzip se complementa con el comando bunzip2 que lo que hace es descomprimir archivos comprimidos con bzip2.

Sintaxis: bzip2 [-parametros] fichero

Parámetros

- d Descomprime un archivo comprimido con bzip2.
- t Comprueba que un archivo comprimido con gzip este correcto. Si se encuentra bien no muestra ningún error y si lo encuentra nos informa de cual es el problema.
- v Muestra el nombre y el porcentaje de reducción por cada fichero comprimido o descomprimido.

Si queremos comprimir un archivo con bzip:

```
bzip2 documento
Esto crearía el archivo documento.bz2. El archivo documento desaparece y se crea documento.bz2 que es el archivo comprimido.
```

Ahora para descomprimirlo habría que hacer lo siguiente:

```
bzip2 -d documento.bz2
```

O también:

```
bunzip2 documento.bz2
```

Esto descomprimiría el archivo documento.bz2 y crearía el archivo documento.

Al usar cualquiera de los dos comandos al comprimir se genera un fichero .gz y desaparece el archivo de origen, al descomprimirlo se crea el fichero comprimido y desaparece el archivo .gz.

A la hora de comprimir se obtiene mayor ratio de compresión comprimiendo con bzip2.

1.2.3 Comando TAR

El comando tar permite crear a partir de varios ficheros un único archivo que los contiene. También permite empaquetar en un único fichero uno o varios directorios.

Sintaxis:

```
tar [-parametros] nombre_archivo.tar [nombre_carpeta_empaquetar]
```

Parámetros

- c crear un archivo
- C DIR cambia al directorio DIR
- x extraer de un archivo
- t listar los contenidos de un archivo
- v ver un reporte de las acciones a medida que se van realizando
- f empaquetar contenidos de archivos
- z Comprime el archivo tar con gzip.
- j Comprime el archivo tar con bzip2.

Como se puede ver, con las opciones z y j se puede comprimir en el mismo paso en el que se empaqueta, lo que puede hacer las cosas más rápidas y cómodas. De todos modos, tar simplemente hace el empaquetado y es gzip el que realiza la compresión. Simplemente que nosotros no tenemos que llamar a gzip, sino que ya lo hace directa e internamente tar.

Si quisieramos hacer un paquete de la siguiente estructura de directorios

```
directorio1/directorio2/directorio
```

tendríamos que usar el siguiente comando:

```
tar vcf directorioEmpaquetado.tar directorio1
```

Esto nos pondría en el archivo directorioEmpaquetado.tar toda la estructura de directorios a partir del directorio1 con todos sus archivos.

A diferencia de gzip y bzip2 no desaparece el archivo o directorios que empaquetemos.

Para desempaquetar el archivo que acabamos de crear:

```
tar vxvf directorioEmpaquetado.tar
```

Con esta orden desempaquetaríamos el archivo y nos generaría de nuevo la estructura de directorios. Al desempaquetar el archivo .tar no se borra el archivo .tar.

El comando tar se suele usar generalmente para hacer backups y se suele combinar con los comandos gzip y bzip2 para que el archivo que se genere ocupe menos espacio. Para empaquetar y comprimir un directorio se puede hacer en dos pasos empaquetando primero el directorio y luego comprimiéndolo o bien en un solo comando.

Empaquetar y comprimir con tar y gzip

Para usar tar y gzip en un solo comando:

```
tar cfvz directorioEmpaquetado.tar.gz directorio
```

Para desempaquetarlo y descomprimirlo:

```
tar xfvz directorioEmpaquetado.tar.gz
```

Si quisieramos desempaquetarlo y descomprimirlo a un directorio concreto:

```
tar zxvf directorioEmpaquetado.tar.gz -C directorioDestino
```

Si queremos ver el contenido de un fichero con extensión tar.gz:

```
tar ztvf directorioEmpaquetado.tar.gz
```

Empaquetar y comprimir con tar y bzip2

Para empaquetar y comprimir con bzip2:

```
tar jfvc directorioEmpaquetado.tar.bz2 directorio
```

Para desempaquetarlo y descomprimirlo:

```
tar jfvx directorioEmpaquetado.tar.bz2
```

Si quisieramos desempaquetarlo y descomprimirlo a un directorio concreto:

```
tar jxvf directorioEmpaquetado.tar.bz2 -C directorioDestino
```

Para ver el contenido de un fichero con extensión tar.bz2:

```
tar jtvf directorioEmpaquetado.tar.bz2
```


2 Comandos para la gestión de procesos

2.1 Procesos en Linux.

Linux, al igual que cualquier sistema Unix, es un **sistema operativo multitarea**. Esto quiere decir que es un sistema operativo capaz de realizar varios “trabajos” al mismo tiempo. A estos “trabajos” se les denomina procesos.

Un **proceso es**, básicamente y a grandes rasgos, **un programa en ejecución**, donde cada proceso tiene asignada una zona de memoria y tiene acceso a los recursos del sistema. Los programas son archivos con código ejecutable que se encuentra almacenado en el disco. Cuando ejecutamos este código, se lleva una copia de él a memoria y se pone en ejecución. A este código en memoria que se está ejecutando se le llama proceso. Un proceso se ejecuta en el procesador con unas características propias y con autonomía respecto al resto de procesos en memoria.

Además, Unix/Linux es un **sistema operativo multihilo**, lo que permite que un proceso pueda tener varios hilos de ejecución, por lo que puede realizar varias cosas al mismo tiempo.

En un sistema Linux, **todo proceso tiene asociado un conjunto de información** que es lo que le permite al sistema operativo realizar todas las tareas relacionadas con la administración de procesos. Esta información es la siguiente:

- **Información general.** Bajo este concepto se almacena, además de la información estadística del proceso, información para identificar tanto al proceso en sí como a su propietario. Destacamos los siguientes campos de información:
 - **Identificador de proceso** (PID – Process IDentification number). Es el número que identifica unívocamente al proceso frente al sistema. Será lo que se tenga que utilizar cuando se quiera referenciar el proceso en algún comando del sistema.
 - **Identificador del proceso padre** (PPID – Parent’s Process IDentification number). Es el PID del padre del proceso. En Linux, todo proceso tiene un proceso padre que es el que lo crea y da vida, aunque posteriormente pueden llegar a ser procesos totalmente independientes.
 - **Usuario propietario del proceso** (UID – User IDentification number). Es el UID del usuario que lanza el proceso. El usuario propietario de un proceso y el administrador del sistema son los únicos usuarios capacitados para modificar dicho proceso (por ejemplo, cambiarle la prioridad, pasarlo a primer o segundo plano, acabar con él, etc.) Además, el sistema se basará en este dato para establecer a qué tiene acceso dicho proceso, que vendrá dado por los permisos que tenga dicho usuario en el sistema.
 - **Grupo propietario del proceso** (GID – Group IDentification number). Es el GID activo del usuario que lanza el proceso.
- **Información de ambiente.** Bajo este concepto se almacena la información sobre el entorno en el que se ejecuta el proceso. Entre esta información destacamos la siguiente:
 - **Directorio actual.** El proceso mantiene actualizado cual es su directorio de trabajo.
 - **Variables de ambiente globales.** Son las variables de entorno con sus valores correspondientes en el momento en el que se lanzó el proceso. Estas variables son heredadas del proceso padre.
 - **Variables de ambiente locales.** Son variables de entorno propias del proceso y que sólo existen en el proceso que las define.
 - **Terminal de control.** Es el terminal desde el que se lanzó el proceso. Esta información le sirve al proceso para establecer desde dónde tiene que recibir la información entrante y dónde tiene que mostrar la información resultado. Una excepción a esto son los procesos demonio o daemons. Éstos son procesos, generalmente de sistema, que una vez lanzados se desvinculan de su terminal de control y se siguen ejecutando inclusive después de cerrada la sesión de usuario desde la cual se lanzaron a correr.
- **Información de Entrada/Salida.** Un proceso mantiene también información acerca de, por ejemplo, los ficheros que está utilizando, así como del resto de dispositivos de entrada/salida.
- **Información de memoria.** En la mayoría de los sistemas multitarea, cada proceso tiene la ilusión de disponer para sí del espacio de direcciones de memoria completo del procesador. En realidad el procesador ve un espacio de direcciones virtual, dividido en páginas. En un instante dado una página puede estar residiendo en la memoria física del procesador o puede estar almacenada en disco. El sistema operativo, con el auxilio del hardware, mantiene al día una tabla con el estado de cada página de memoria del proceso.

Un proceso puede encontrarse en alguno de los siguientes estados:

- **En ejecución.** Un proceso en ejecución puede encontrarse en modo usuario, que es su estado normal, o en modo núcleo, que es cuando se encuentra ejecutando alguna llamada al sistema.
- **Listo.** Son los procesos que están listos para ser ejecutados y simplemente están esperando a que se les conceda el uso de la CPU.
- **Dormido (Bloqueado).** Se llama así a los procesos que se encuentran esperando que se les de acceso a un recurso compartido o esperando los resultados de una operación de Entrada/Salida.
- **Parado o detenido.** Son los procesos a los que se les ha mandado una señal (generalmente el propio usuario pulsando CNTRL+Z) para que dejen de ejecutarse. Cuando se pone un proceso en este estado, está prohibida su ejecución, por lo que deja de ser considerado a efectos de planificación para acceder a la CPU)
- **Suspendido.** Son los procesos que se han llevado a disco completamente.
- **Zombie.** Son los procesos que han acabado su ejecución y que están esperando a que su proceso padre los termine definitivamente.

En un sistema multitarea, donde hay varios procesos compitiendo por hacer uso de la CPU, debe haber un mecanismo para administrar el recurso y decidir en cada momento cuál es el proceso que debe ejecutarse.

Linux utiliza un algoritmo de planificación basado en la prioridad de los procesos. El sistema dispone de varias colas, cada una asociada a un nivel de prioridad y en cada instante será el proceso con mayor prioridad el que acceda a la CPU. Para ello, el planificador toma el primer proceso de la cola de mayor prioridad que no esté vacía. Este proceso estará ejecutándose hasta que se bloquee o termine o hasta que agote su quantum. Podemos observar, por tanto, que **el algoritmo de planificación que utiliza Linux no es un algoritmo por prioridad puro, sino un híbrido entre el algoritmo por prioridad y el Round Robin**: procesos con igual prioridad comparten la CPU con un algoritmo Round Robin. Cuando un proceso agota su quantum, se le coloca al final de su cola y se vuelve a elegir otro proceso al que darle el control de la CPU ejecutar.

Además, este sistema de planificación por prioridad utiliza prioridades dinámicas. **Un proceso tiene una prioridad base o número nice, que es siempre la misma si no es cambiada por el propio usuario o por el administrador** y que sirve para definir la calidad de servicio (prioridad para acceder a la CPU) que se desea obtener para el proceso. **Cuanto menor sea el número mayor será la prioridad del proceso.** En Linux, los números nice van desde -20 hasta 20, siendo -20 el número que representa la prioridad mayor. El número nice 20 se le pone a aquellos procesos que se quiera que sólo accedan a la CPU cuando nadie más quiera hacerlo. Normalmente, la prioridad base de todo proceso es 0. **Un usuario puede pedir un servicio “peor” aumentando su número nice y haciéndolo positivo. Sólo el superusuario puede pedir un servicio “mejor”; es decir, hacer que su número nice sea negativo.** Además, un usuario normal, aparte de no poder tener un número nice negativo, no puede disminuir su número nice una vez establecido.

Sin embargo, **a la hora ser considerado por el algoritmo de planificación, lo que se considera no es esta prioridad base o número nice, sino una prioridad dinámica** que, si bien es cierto que se basa en esta prioridad base, también considera otros aspectos, como el uso de la CPU que se ha hecho recientemente. De esta manera, un proceso que ha utilizado mucho la CPU de manera reciente ve penalizada su prioridad, mientras que aquellos procesos que no han disfrutado de CPU ven aumentada su prioridad dinámica y ganan “puntos” para acceder a ella. La prioridad dinámica de los procesos se recalcula constantemente siguiendo la siguiente fórmula:

$$\begin{aligned} \text{UsoCPU} &= \text{UsoCPU} / 2 \text{ Para no castigar eternamente a los procesos que han utilizado la CPU.} \\ \text{Nueva Prioridad} &= \text{PrioridadBase} + \text{UsoCPU} \end{aligned}$$

Este algoritmo de planificación lo que pretende es, por un lado, y al igual que todos los algoritmos de planificación, ser justo con los diferentes procesos. Además, también pretende dar buena respuesta a los procesos interactivos. Por otra parte, **Linux aumenta la prioridad de los procesos cuando éstos se encuentran en modo núcleo**; es decir, cuando se encuentran realizando una llamada al sistema. **Los procesos del administrador suelen tener más prioridad que los procesos de los usuarios, principalmente debido a que son procesos que pueden tener un número nice negativo.**

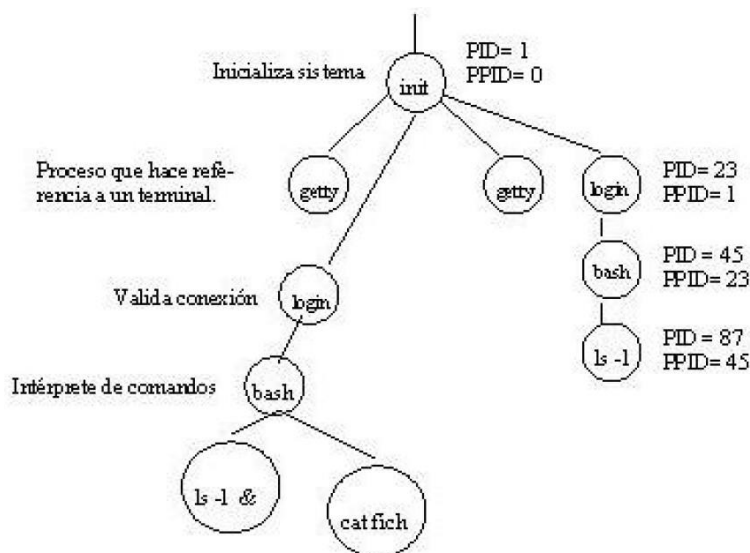
Como ya se ha comentado, todo proceso en Linux es hijo de otro proceso, aunque una vez creado puede llegar a ser un proceso totalmente independiente de su padre. Se plantea entonces la siguiente pregunta: ¿cuál es el primer proceso del sistema y cómo se crea? **Durante el arranque del sistema operativo Linux, se carga en memoria el kernel del sistema**, que estará activo mientras el sistema esté levantado. **Una de las cosas que realiza este kernel al ser cargado en memoria es crear y lanzar este primer**

proceso que tiene PID 1, PPID 0 y que se llama init. Este proceso es el encargado de levantar el resto del sistema para ponerlo en funcionamiento.

Hay dos formas de crear y lanzar procesos desde el shell:

- **Primer plano.** El shell espera a que termine el proceso hijo que acaba de lanzar para que ejecute el comando antes de proseguir con su tarea de recibir y ejecutar otros comandos. Es decir, hasta que no se termine de ejecutar el comando que se acaba de lanzar el shell no mostrará de nuevo el prompt.
- **Segundo plano,** también conocido como **background.** El shell no espera a que el proceso creado finalice para poder pedir la introducción y ejecución del siguiente comando. Es decir, el shell mostrará el prompt de sistema nada más lanzar el comando y el usuario podrá ejecutar otro mandato de forma inmediata. Para ejecutar un comando o programa como un proceso en segundo plano habrá que ejecutar el mandato seguido del símbolo &. El shell notifica el PID asignado a ese proceso para que podamos hacer referencia a él posteriormente. Podemos aprovechar esta posibilidad de lanzar procesos en segundo plano cuando tengamos que ejecutar comandos que necesiten mucho tiempo para finalizar y no queramos dejar bloqueado el terminal. Un inconveniente es que cuando el proceso en segundo plano acabe mostrará los resultados por pantalla en ese instante y puede entorpecer el trabajo que estamos realizando en ese momento. Para evitarlo, lo único que podemos hacer es ejecutar el comando en segundo plano redireccionando las salidas a algún fichero en vez de a la salida estándar.

Toda esta estructura de procesos hasta llegar a los procesos de usuario viene expresada en la siguiente figura:



2.2 Redireccionamiento.

Un proceso Linux dispone de una entrada estándar y dos salidas estándares (una normal y otra de errores). Todos los programas tienen estos tres ficheros estándares, creados cuando empiezan a ejecutarse (pasan a ser procesos) y numerados con enteros pequeños, llamados descriptores de ficheros estándar, y son :

0	Entrada estándar	(teclado del terminal)
1	Salida estándar	(pantalla del terminal)
2	Salida de error estándar	(pantalla del terminal)

Como es conocido, cada uno de los ficheros de entrada y salida estándar pueden ser redireccionados. Incluso la salida de error, en lugar de a la pantalla del terminal, puede ser enviada a un fichero, mediante la siguiente construcción:

`ls /noexiste 2> ferror` (no hay espacio entre el 2 y el >)

Al no encontrar el directorio especificado, se generará un mensaje de error que se visualizaría por la pantalla mediante la salida estándar de error (2). Redireccionando esta salida hacia el fichero `ferror` obtendremos un fichero de errores.

Existe un dispositivo que se llama `/dev/null` al que se pueden redireccionar todo aquello que no interesa registrar ni siquiera por pantalla. Este es un dispositivo nulo o nada.

Ejemplo:

```
ls -R /home 2> /dev/null
```

Los directorios con acceso permitido aparecerán por pantalla y los otros no.

2.3 Comandos sobre procesos.

2.3.1 Comando PS

Este es el comando principal para obtener información de los procesos en ejecución. Mostrará una instantánea del estado del sistema en el momento de la ejecución del comando, en cuanto a procesos se refiere. La lista de los procesos así como su información asociada se mostrará en forma tabular.

Sintaxis: `ps [opciones]`

Parámetros

Si no se usa ninguna opción, muestra información acerca de los procesos que se están ejecutando en ese momento en su terminal y que le pertenecen.

- a Muestra información de los procesos asociados a un terminal (el que sea), le pertenezcan o no. Combinado con la opción `x` (`ps ax`) muestra todos los procesos del sistema.
- x Muestra información de los procesos que te pertenecen. Combinado con la opción `a` (`ps ax`) muestra todos los procesos del sistema.
- A Muestra todos los procesos
- e Muestra todos los procesos
- u Muestra información de los procesos con un formato orientado al usuario. Deberá usarse con otras opciones, por ejemplo, `ps au`, `ps xu`, etc.
- l Muestra información de los procesos con un formato de listado largo. Deberá usarse con otras opciones, por ejemplo, `ps al`, `ps xl`, etc. (con esta opción no se puede usar la opción `u`).
- w Muestra información de los procesos con un formato ancho. Usa esta opción dos veces para tamaño ilimitado.
- h No muestra la cabecera.
- H Muestra la jerarquía de procesos pues los hijos aparecen debajo de su padre un poco tabulados
- f Muestra la jerarquía de procesos con caracteres ASCII en vez de tabulaciones
- c Muestra el verdadero nombre del comando en vez de el comando con los argumentos con los que se invocó.
- o `format` Muestra los campos que se indiquen en “format” separados por comas. Para saber cómo se indican los campos a mostrar, véase la página de manual, en el apartado “STANDARD FORMAT SPECIFIERS”
- t `xx` Sólo muestra los procesos asociados al terminal `xx`.
- T Sólo muestra los procesos asociados a “este” terminal.
- C `comando` Sólo muestra los procesos cuyo ejecutable venga dado por “comando”. “comando” puede ser una lista de comandos separadas por comas.
- G `grupo` Sólo muestra los procesos cuyo grupo real es “grupo”. “grupo” puede ser una lista de usuarios separadas por comas.
- g `grupo` Sólo muestra los procesos cuyo grupo efectivo es “grupo”. “grupo” puede ser una lista de usuarios separadas por comas.
- U `user` Sólo muestra los procesos cuyo usuario real es “user”. “user” puede ser una lista de usuarios separadas por comas.
- u `user` Sólo muestra los procesos cuyo usuario efectivo es “user”. “user” puede ser una lista de usuarios separadas por comas.
- p `PID` Sólo muestra los procesos cuyo PID sea “PID”. “PID” puede ser una lista de PIDs

Los campos más importantes que pueden aparecer en las distintas ejecuciones de este comando, tienen el siguiente significado: (Para saber otros campos a mostrar, véase la página de manual, en el apartado “STANDARD FORMAT SPECIFIERS”):

- *USER*. Nombre del usuario propietario del proceso.
- *PID*. Identificador del proceso.
- *PPID*. Identificador del proceso padre del proceso.

- **%CPU.** Porcentaje del tiempo que ha ocupado de CPU con respecto al tiempo que lleva en memoria.
- **%MEM.** Porcentaje o fracción estimada de la memoria consumida.
- **RSS.** Memoria real usada. El número de kilobytes del programa que están residiendo en la memoria en ese momento.
- **TTY.** Terminal asociado al proceso. Una ? en este campo indica que el proceso no tiene Terminal asociado, algo que pasa por ejemplo con los procesos encolados, demonios o procesos en segundo plano cuyo shell ha muerto.
- **STAT.** Estado actual del proceso, donde:
 - R = proceso ejecutándose o preparado para ejecutarse.
 - S = proceso bloqueado o durmiendo que está esperando un evento para volver a estar listo para ejecutarse.
 - D = proceso bloqueado por espera de una E/S.
 - T = proceso parado.
 - X = proceso muerto (este estado nunca debe aparecer)
 - Z = proceso zombie. Es un proceso que ha terminado su ejecución pero al que el padre no ha terminado de destruir pues ha muerto. Es un proceso que no consume recursos pero que no puede ser eliminado.
 - N = prioridad del proceso mayor que 0.
 - < = prioridad del proceso menor que 0.
 - s = es un líder de sesión
 - l = es un proceso multihilo
 - + = es un proceso que está ejecutándose en primer plano
- **START.** Hora a la que empezó el proceso.
- **TIME.** Tiempo en segundos consumidos de CPU.
- **COMMAND.** Línea de comando que fue ejecutada.
- **PRI.** Prioridad dinámica actual del proceso.
- **NI.** Número nice del proceso o prioridad estática.
- **VSZ.** Tamaño virtual del proceso.
- **WCHAN.** Evento que un proceso está esperando en modo núcleo.

2.3.2 Comando PSTREE

Muestra un árbol con los procesos en ejecución. Es muy útil para ver las relaciones de paternidad entre procesos.

2.3.3 Comando NICE

Este comando se utiliza para lanzar un proceso con un número nice (prioridad estática) distinto al número nice del proceso padre. Generalmente, el número nice del proceso shell de un usuario normal es 0, por lo que todos los procesos que lance dicho usuario tendrán dicha prioridad.

Sintaxis: `nice -numeroNice comando`

Un usuario sólo puede utilizar como *numeroNice* números positivos (decrementar la prioridad del proceso). Tan sólo el superusuario puede utilizar como *numeroNice* un número negativo de la forma, por ejemplo: `nice --3 comando`

2.3.4 Comando RENICE

Este comando se utiliza para variar el número nice (prioridad estática) de un proceso que ya ha sido lanzado.

Sintaxis: `renice nuevoNumeroNice PID`

Parámetros

- g El parámetro PID es interpretado como un GID y se aplicará el renice a todos los procesos cuyo GID sea el especificado
 - u El parámetro PID es interpretado como un UID y se aplicará el renice a todos los procesos cuyo UID sea el especificado.
 - p El parámetro PID es interpretado como un PID y se aplicará el renice al mismo
- Un usuario normal sólo puede utilizar este comando con los procesos que les pertenezcan. **El administrador puede cambiar la prioridad a cualquier proceso.** Un usuario no puede poner a sus procesos una prioridad negativa, mientras que **el superusuario sí.**

Ejemplos:

```
renice +3 PID
renice -5 PID
```

Pueden usarse varios de los parámetros, y cada parámetro puede tener varios argumentos.

Ejemplo:

```
renice +1 987 999 -u user1 user2 -g group1 group2 -p 32 45
```

Este comando establece la prioridad a +1 a los procesos **987**, **999**, **32** y **45**, y también a todos los procesos de los usuarios **user1** y **user2** y a todos los procesos de los grupos **group1** y **group2**

2.3.5 Comando JOBS

Muestra los procesos asociados al terminal y que se encuentran ejecutándose en segundo plano.

Sintaxis: jobs

Cada uno de los procesos que se muestran tiene asociado un número, que viene entre corchetes. Además, sobre cada proceso se muestra su estado (Running, Stopped) y el comando que se utilizó para lanzarlo.

Ejemplo:

```
[1] Running find / -name lo* &
[2] Running ./tetris &
```

2.3.6 Comando FG

Cuando un proceso se encuentra ejecutándose en segundo plano, podemos traerlo a primer plano para mandarle una señal. Esto puede hacerse mediante el comando fg.

Sintaxis: fg numeroJobs

Este comando trae al primer plano el proceso que en el comando jobs viene identificado con el número *numeroJobs* que se pasa como parámetro. Una vez que el programa se encuentra en primer plano podemos mandarle una señal. Así, por ejemplo, si pulsamos CNTRL+Z pondremos el proceso en estado “parado” o “detenido” o “stopped” y si pulsamos CNTRL+C “mataremos” el proceso.

2.3.7 Comando BG

Este comando vuelve a reanudar en segundo plano un proceso que ha sido detenido.

Sintaxis: bg numeroJobs

Este comando reanuda en segundo plano el proceso que en el comando jobs viene identificado con el número *numeroJobs* que se pasa como parámetro.

2.3.8 Comando KILL

Manda una señal a un proceso. Normalmente suele ser utilizado para detener o matar a un proceso en ejecución.

Sintaxis: kill [-señal] PID

El comando kill -l muestra una lista de todas las señales disponibles en el sistema. Entre las señales más importantes que se pueden mandar a un proceso tenemos las siguientes:

Número	Nombre	Descripción
1	SIGHUP	Cuando un terminal se cuelga, se envía a cada proceso cuyo terminal de control es el que se ha quedado colgado. También se envía a cada proceso de un grupo cuando el líder del grupo finaliza su ejecución.
2	SIGINT	Le manda al proceso la señal de interrupción. Es la misma señal que se le manda a un proceso cuando se pulsan las teclas CNTRL+C.
9	SIGKILL	Matar un proceso. Obliga al proceso a terminar de manera inmediata. El proceso ni siquiera cierra los ficheros abiertos, no libera su memoria (esto ya lo hará el sistema operativo), etc. En definitiva, es una terminación no limpia de un proceso. Esta señal no puede ser ignorada por el proceso.
15	SIGTERM	Finalizar el proceso de manera controlada y limpia. Suele usarse antes de usar la señal SIGKILL permitiendo al proceso prepararse para ser finalizado (cerrar sus ficheros abiertos, liberar la memoria, etc.)
18	SIGCONT	Continuar después de una señal SIGSTOP. Esta señal no puede ser ignorada.
19	SIGSTOP	Detener el proceso.

Un usuario sólo puede mandar señales a procesos que le pertenezcan, por lo que sólo podrá matar sus propios procesos. **El superusuario puede mandar señales a cualquier proceso.** Cuando se mata a un proceso también se mata a todos sus procesos hijo.

No todos los procesos mueren. Hay una serie de procesos que se niegan a morir pese al `kill`, al encontrarse en modo núcleo. Estos son: No todos los procesos mueren. Hay una serie de procesos que se niegan a morir pese al `kill`, al encontrarse en modo núcleo. Estos son:

- Los procesos zombies.
- Los procesos que se encuentran esperando un recurso via NFS (a través de la red) que no está disponible.
- Los procesos que están esperando que un dispositivo termine una operación, por ejemplo una entrada/salida de disco.

3 Bibliografía

GONZALO FERNÁNDEZ HERNÁNDEZ Apuntes de DSFI de 2º ASI.

FRANCISCO JAVIER MUÑOZ LÓPEZ. Sistemas operativos monopuesto. McGraw-Hill, 2009

GUÍA UBUNTU www.guia-ubuntu.org/